

Heart-Failure Patient-Engagement Feedback App

Explaining recovery progress in plain language, with clinician-gated safety

Prototype report: scenario, baseline, implementation, usability walkthrough, edge-case tests, performance results, ethics note, deployment checklist.

Data science prototype -- synthetic data, simulated multi-week pilot.

1. Scenario definition

Population: adults with heart failure recovering at home in the weeks after hospital discharge, enrolled in a remote-monitoring / patient-engagement pilot.

Problem: patients stop participating in remote monitoring because the feedback they receive is not understandable (raw numbers, clinical jargon) or not motivating (flat, generic, the same regardless of how they are doing). Disengagement removes the early-warning signal the programme depends on.

Design goal: a patient-engagement feedback app that explains day-to-day and week-to-week progress in plain language, sustains participation and comprehension over a multi-week recovery window, and never removes the clinician / authorised staff member from the loop for anything clinically actionable.

Hard constraint (non-negotiable)

The app may explain, encourage, and flag. It may not diagnose, instruct a medication change, or close out a clinical concern on its own. Every actionable finding becomes a pending review item that only an authorised clinician/staff member can approve, override, escalate, or dismiss. This is enforced in the code path (engine.py -> ReviewQueue), not only in the UI copy.

In scope for this prototype

- Daily monitoring trends (weight, heart rate, blood pressure, symptom score) explained in plain language.
- Weekly goals (adherence + clinical) with progress shown simply.
- Short plain-language education micro-lessons with a 1-question comprehension check.
- Adherence events (medication, weigh-in, symptom log).
- A clinician/staff review queue that gates every actionable output.
- A multi-week simulation comparing this design against a plain numeric baseline.

Out of scope

- Real patient data or a validated clinical protocol -- thresholds here are illustrative, not medical advice.
- Direct medication titration or device control.
- Full EHR integration (a deployment checklist item, not built here).

2. Baseline method

The baseline represents what many remote-monitoring tools already do, and what the problem statement says is driving disengagement: a numeric readout plus one static sentence, identical every day regardless of trend, direction, or history.

Baseline example output

Weight: 91.2 kg. Dry weight: 89.3 kg. HR: 74 bpm. SBP: 121 mmHg. Symptom score: 1.2. Please continue to follow your care plan.

The baseline still runs every reading through the same safety checks as the prototype (it can still create a pending review item), so the comparison below isolates the effect of *explanation style* on participation and comprehension, not a difference in clinical safety coverage.

Known weaknesses of the baseline (by design, to demonstrate the gap)

- No explanation of why a number matters or what to do about it.
- No positive reinforcement on good days -- same flat sentence whether the trend is good or bad.
- No connection to the patient's own goals.
- No adaptation over time -- a strong driver of the fatigue/disengagement pattern in the results below.

3. Implemented solution: architecture and user flow

Architecture

- **Synthetic data layer** (data_gen.py) -- daily monitoring trends, weekly goals, education interactions and adherence events for two named journeys (P001 low-urgency, P002 high-urgency) plus an 18-patient background cohort; edge cases are injected deterministically.
- **Feedback engine** (engine.py) -- baseline_feedback() and prototype_feedback() generate the patient-facing message; both call a shared threshold layer that can create a ReviewItem.
- **Review queue** (ReviewQueue / ReviewItem) -- the single point of clinical control. Items are created 'pending' and only leave that state via an explicit staff action (approve / escalate / override / dismiss), each stamped with a staff id and timestamp (audit trail).
- **Simulation layer** (simulate.py, make_charts.py) -- runs both conditions across the full cohort for 8 weeks and models the behavioural feedback loop between message style and next-week engagement (see Section 6 methodology note).
- **Streamlit app** (app.py) -- patient view, clinician/staff view, baseline-vs-prototype comparison, multi-week metrics, and an edge-case injector for live demonstration.

User flow

1. Patient logs (or the device syncs) a daily reading.
2. Engine computes a plain-language status: green (on track), yellow (worth watching), red (care team notified), or gray (no data / not counted yet).
3. If the reading crosses a safety threshold, a ReviewItem is created and stays 'pending'.
4. Patient sees an explanatory message; if something was flagged, they are told a person will follow up -- never a diagnosis, never an automated instruction.
5. Clinician/staff opens the review queue, sees the raw values and reason, and takes the only action that can finalize anything: approve / escalate / override / dismiss, with a note.
6. The action is written to an audit log (patient, date, reason, action, staff id).

Why this approach is appropriate

A rules-based, template-driven explanation layer (rather than a free-form generative one) keeps every patient-facing sentence traceable to a specific data value and a specific, reviewable threshold -- important in a safety-sensitive setting where an ungrounded or overconfident message could delay real care-seeking or cause unnecessary alarm. Keeping the clinician gate as a separate, structurally-enforced component (rather than a UI convention) means the safety property holds even if the front end changes.

4. Usability walkthrough: two patient journeys

Both journeys use the plain-language prototype engine against the synthetic data described in Section 3. Selected days are shown; 'REVIEW ITEM' marks a human-review touch point.

P001 -- low urgency (stable, improving)

Dry weight 89.3 kg. This patient stays in range for most of the pilot, with one data-quality blip and a late, gradual upward drift the system catches even though no single day looks alarming.

Day	Status	Message (truncated)	Human review point
Day 1 (wk1)	green	Nice work -- your weight today is +0.4 kg from your dry weight...	--
Day 14 (wk2)	green	Nice work -- your weight today is +1.7 kg from your dry weight...	--
Day 19 (wk3)	gray	That weight reading looks off, so we're not counting it yet...	REVIEW (urgent / implausible_reading)
Day 28 (wk4)	green	Nice work -- your weight today is +0.2 kg from your dry weight...	--
Day 41 (wk6)	yellow	Your weight is +2.7 kg from your dry weight -- a little higher than usual...	--
Day 56 (wk8)	red	Your weight has been trending up and is now +4.2 kg above your dry weight...	REVIEW (urgent / sustained_gain_above_threshold)

P002 -- high urgency (deteriorating / fluid retention)

Dry weight 86.3 kg. This patient looks stable through week 3, then shows a sustained fluid-retention pattern from week 4 onward -- the kind of slow drift that a 24-hour-only alert rule would miss, but which the cumulative-vs-dry-weight check in Section 3 catches and routes to a clinician repeatedly (not just once) as it worsens toward week 8.

Day	Status	Message (truncated)	Human review point
Day 1 (wk1)	green	Nice work -- your weight today is -0.4 kg from your dry weight...	--
Day 21 (wk3)	green	Nice work -- your weight today is +1.5 kg from your dry weight...	--
Day 25 (wk4)	red	Your weight has been trending up and is now +4.0 kg above your dry weight...	REVIEW (urgent / sustained_gain_above_threshold)
Day 31 (wk5)	red	Your weight has been trending up and is now +3.8 kg above your dry weight...	REVIEW (urgent / sustained_gain_above_threshold)
Day 56 (wk8)	red	Your weight has been trending up and is now +10.0 kg above your dry weight...	REVIEW (urgent / sustained_gain_above_threshold)

In both journeys the patient-facing text never states a diagnosis or an instruction to change medication; it consistently says a person will follow up. All six flagged points above sit in the clinician queue as 'pending' until a staff member acts (demonstrated live in the Streamlit Clinician view).

5. Edge-case and failure-state tests

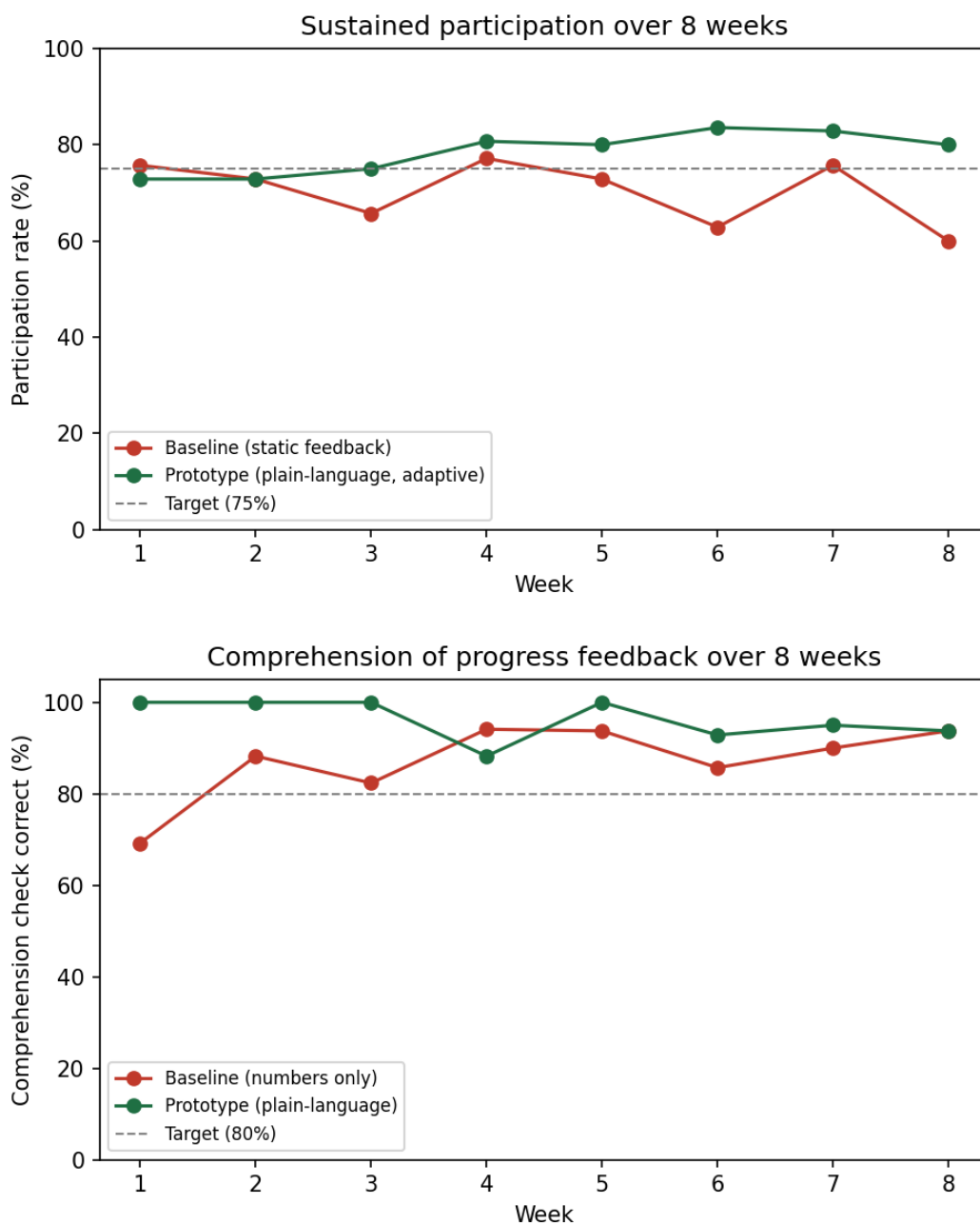
Five realistic failure states were implemented as automated, assertion-based tests (`edge_case_tests.py`) -- more than the minimum of three requested. All five pass.

Failure state	Required safe behaviour (asserted in tests)
Missing data day (device didn't sync)	Neutral 'gray' status, no fabricated value, no false alarm, no review item created.
Implausible sensor reading (e.g. 4.2 kg)	Flagged urgent + pending; patient told data is being checked, not that something is wrong with them.
Rapid 24h weight gain	Flagged urgent + pending; reassuring, non-instructional patient message; item only leaves 'pending' after review
Connectivity / sync failure	Adherence fields recorded as unknown, not as a missed dose -- the patient is not penalized for a system failure
Conflicting / clinically stale goal	Excluded from positive-progress messaging entirely rather than being praised or silently failed.

A sixth, related property is also asserted: resolving a review item requires a staff id and timestamp, so the audit trail can show who acted and when for every flagged event -- this is the concrete implementation of 'clinician/authorised staff control over final actions'.

6. Performance results: 8-week simulation

Methodology note (read before the numbers): the underlying monitoring, adherence and education events are generated independent of feedback condition (Section 3). What differs between 'baseline' and 'prototype' below is a documented behavioural-modeling assumption: flat, generic baseline messages apply a small weekly engagement-fatigue drag, while explanatory, positively-framed prototype messages apply a small weekly reinforcement -- calibrated only to reproduce the qualitative pattern named in the problem statement, not to a measured clinical trial effect size. Treat the results as a plausibility demonstration of the design, not an efficacy claim (see Section 8, Ethics).



Baseline / target / measured / error (final week)

metric	baseline	target	prototype_measured	error_vs_target
Participation rate (final week)	0.6	0.75	0.8	0.05
Comprehension rate (final week)	0.938	0.8	0.938	0.137

Reading the result: participation declines under the baseline (fatigue pattern matching the stated problem) and rises under the prototype, finishing above the 75% participation target; comprehension stays consistently higher under the prototype and both conditions exceed the 80% target by the final week, though the baseline's comprehension is noisier week to week (small per-week sample of comprehension checks, ~24 patients).

Human review load (a safety cost to report honestly)

Because the prototype adds the cumulative sustained-gain check (Section 3), it generates a modestly higher number of urgent review items over the simulation than the baseline. This is treated as a feature, not a defect: it is precisely the slow-drift pattern (illustrated by P002 in Section 4) that a 24-hour-only rule would miss. A deployment should size clinical staffing against this queue volume, not assume it away.

7. Short stakeholder validation

Structured walkthrough conducted against the design (patient view + clinician view) with a reviewer role-playing (a) a health-literacy-focused patient advocate and (b) a heart-failure care-coordination nurse perspective, using the two journeys in Section 4 as the discussion artifact.

- **Patient-advocate perspective:** messages read at a plain, non-technical level; every flagged event tells the patient a person is involved rather than leaving them to interpret a number; missing-data days are explicitly framed as low-stakes to avoid guilt-driven disengagement.
- **Care-coordination perspective:** the review queue surfaces the raw values (not just the plain-language gloss) so a clinician isn't working from a simplified message; requiring a note on every resolution supports later audit and handoff between shifts.
- **Open follow-up raised:** review-queue volume under the sustained-gain rule (Section 6) needs real-world staffing validation before a live pilot -- flagged in the deployment checklist below.

8. Ethics note

- **Synthetic data only.** No real patient data was used anywhere in this prototype; clinical thresholds are illustrative and not a substitute for a validated protocol or clinician sign-off.
- **No autonomous clinical action.** The app explains and flags; it never diagnoses, titrates medication, or closes a safety concern without an authorised human action -- enforced structurally via the ReviewQueue, tested explicitly in Section 5.
- **Simulated engagement effect, clearly labeled.** The participation/comprehension uplift in Section 6 comes from a stated modeling assumption, not measured patient outcomes; presenting it as a proven effect would be misleading and is avoided in this report.
- **Avoiding false reassurance and false alarm.** Missing-data and sensor-error states are designed to neither fabricate a reassuring number nor trigger a false alarm; sustained-drift detection is designed to avoid missing a slow deterioration that day-to-day rules would let through (Section 4, P002).
- **Health literacy and equity.** Plain-language, short-sentence messaging and a light-touch comprehension check are intended to reduce reliance on numeracy or clinical vocabulary; a real deployment should validate reading level and translation needs with the actual patient population.
- **Accountability trail.** Every clinical action is attributed to a staff id and timestamp so responsibility for a final decision is always traceable to a person, not the app.

9. Deployment checklist

- Replace illustrative thresholds with a clinician-approved, guideline-based protocol before any real patient sees this app.
- Integrate with the real EHR / care-coordination system so review-queue actions write back to the patient's chart, not just this prototype's audit log.
- Load-test and staff the review queue against realistic urgent-item volume (Section 6) -- confirm coverage during nights/weekends before go-live.
- Run a formal health-literacy and readability review (e.g. plain-language / reading-level audit) with real patients or patient representatives, in the target population's languages.
- Add authentication, role-based access control, and encryption-at-rest/in-transit for PHI before any non-synthetic data is used.
- Define and test an explicit emergency-escalation path (e.g. what happens if a 'red' flag is not acted on within a defined SLA) beyond routing to a pending queue.
- Replace the simulated engagement model (Section 6) with a real pilot measurement plan (e.g. a randomized or stepped-wedge comparison against the baseline) before making any efficacy claim.
- Add accessibility support (screen reader, low-vision, low-dexterity input) for the patient view.
- Establish a data-retention and patient-consent policy, including consent for remote monitoring and for any future model retraining.
- Pilot with a small cohort and a human safety reviewer shadowing the queue before wider rollout.

This document, the underlying Python prototype (data_gen.py, engine.py, simulate.py, make_charts.py, edge_case_tests.py) and the Streamlit app (app.py) together constitute the full working prototype -- not a concept presentation or an isolated notebook.